VidyaOS Architecture & API Flow

The main architecture and every layer — now with the exact API calls labeled on each connection.

Guru Nanak Public School, Hambran, Ludhiana

v2.0 · API-annotated · September 2026

Contents

01	System Overview + API paths	6 layers, both data paths
02	Frontend Layer + API calls	which APIs each module calls
03	Backend Layer + endpoints	Path A vs Path B, RLS
04	Database Layer + queries	tables and the calls that hit them
05	Deployment Layer	where each API physically runs

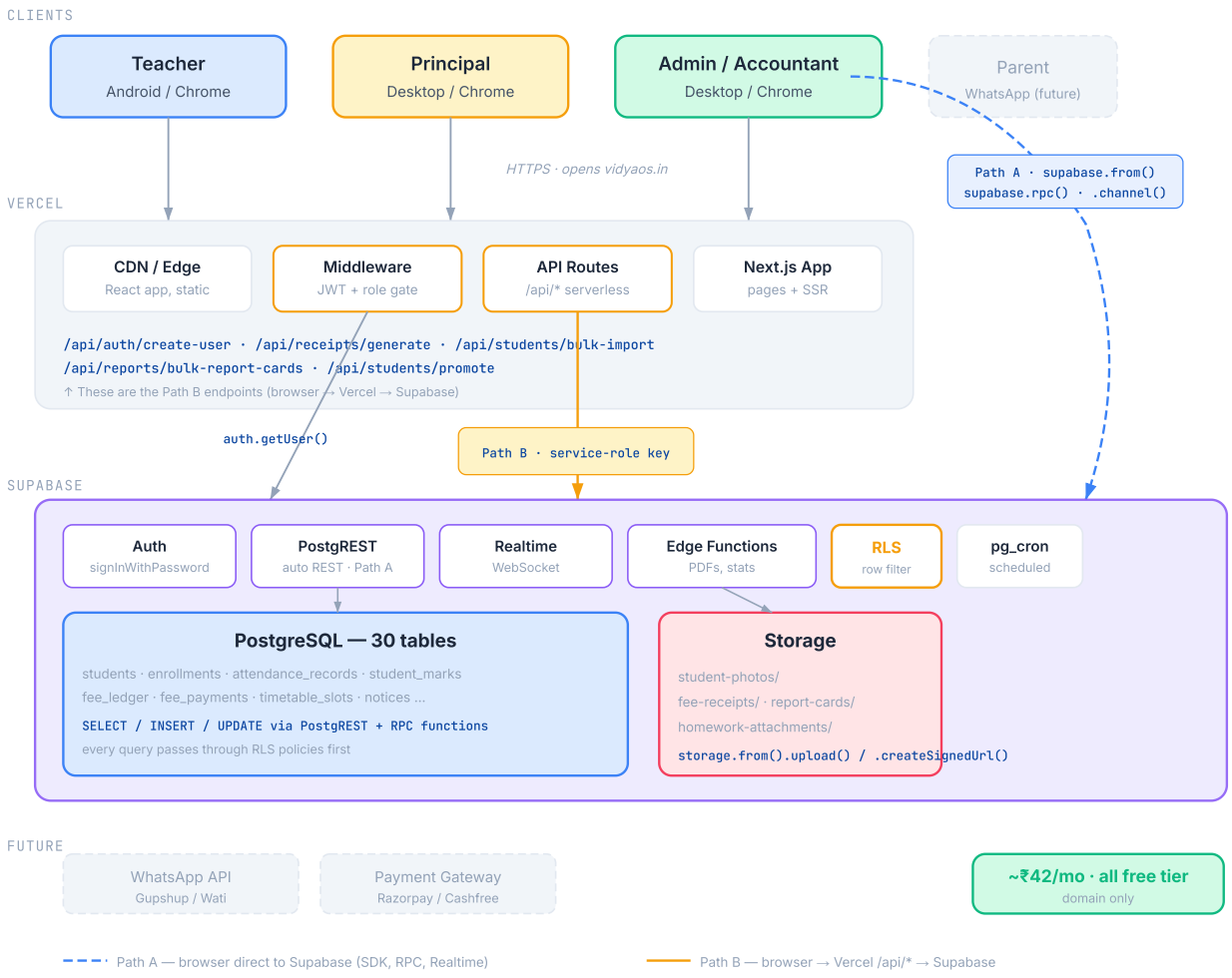
How to read the API pills

Path A	SDK	Browser talks **directly** to Supabase. `supabase.from(...)`. JWT + RLS filter the data.
Path B	API	Browser calls a Vercel serverless route first. `/api/*`. Server uses the service-role key.
RPC	DB FN	A PostgreSQL function called for atomic multi-table work. `supabase.rpc(...)`.
WS	REALTIME	WebSocket subscription for live updates. `supabase.channel(...)`.

DIAGRAM 01

System Overview — with API paths

The 6-layer architecture. Blue lines = Path A (direct SDK). Amber lines = Path B (via Vercel API routes). Each line is labeled with the actual call.



Every API in the system — grouped by path

`supabase.auth.signInWithPassword()` SDK

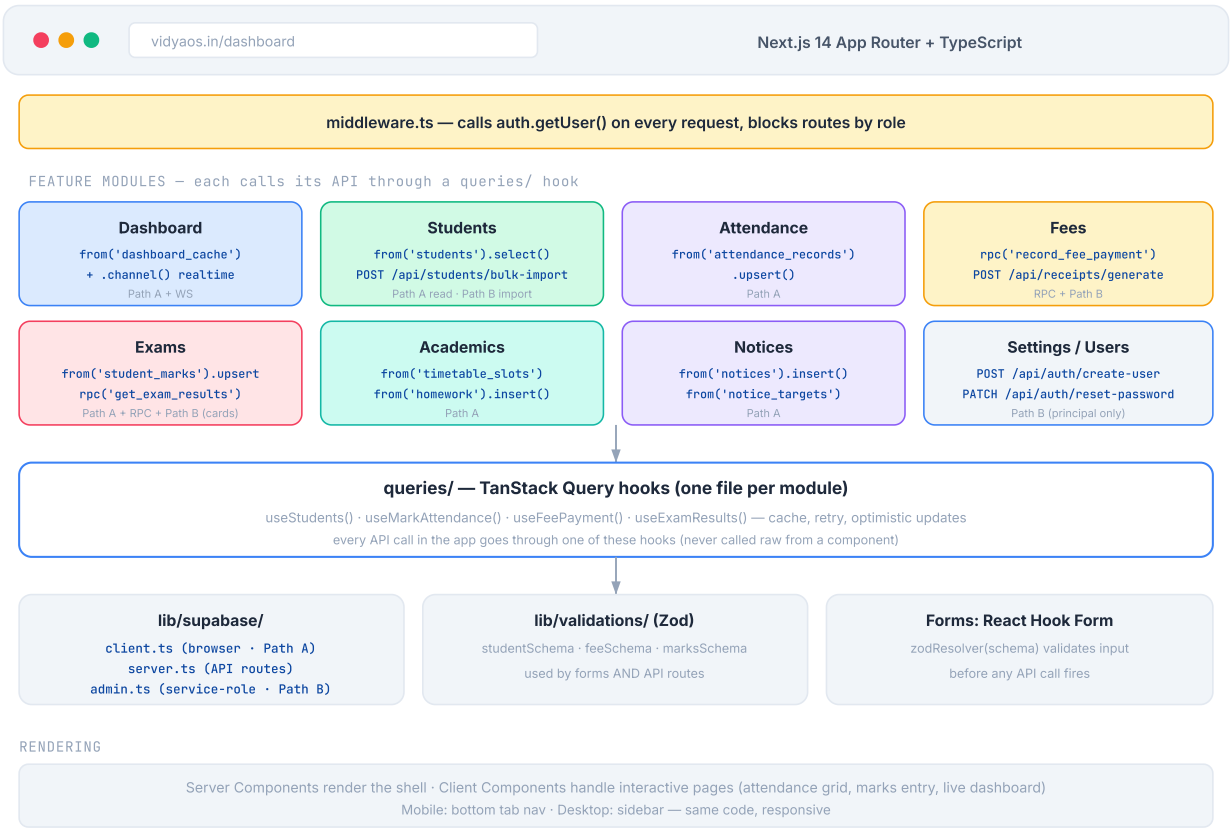
Login. Username → `username@vidyaos.local`. Returns JWT stored in HTTP-only cookie.

`supabase.from('table').select/insert/upsert` SDK	All everyday reads and writes — students, attendance, marks, fees, timetable, notices, homework.
`supabase.rpc('fn_name')` DB FN	Atomic multi-table ops: `record_fee_payment`, `get_exam_results`, `create_student_with_enrollment`.
`supabase.channel().on()` REALTIME	Live dashboard, and job-status updates for bulk report cards.
`supabase.storage.from().upload()` SDK	Photo uploads (browser). Signed URLs for reading private files.
`POST /api/auth/create-user` API	Admin creates a teacher/staff account (needs service-role key — can't run in browser).
`POST /api/students/bulk-import` API	Validated Excel rows → batch insert. Server re-validates and checks duplicates.
`POST /api/students/promote` API	Year rollover — creates new enrollment rows for the next class.
`POST /api/receipts/generate` API	Renders a fee-receipt PDF, uploads to Storage, returns a signed URL.
`POST /api/reports/bulk-report-cards` API	Queues a background job; a Supabase Edge Function generates the PDFs.

DIAGRAM 02

Frontend Layer — which module calls which API

The Next.js app structure. Each feature module is labeled with the API it calls and the path it uses.



Frontend → API mapping (what each screen calls)

Dashboard `from('dashboard_cache').select()` SDK then `.channel('dashboard-live')` REALTIME for live refresh.

Students list `from('students').select(..enrollment..)` SDK. Import: `POST /api/students/bulk-import` API.

Attendance `from('attendance_records').upsert()` SDK — RLS restricts to the teacher's class.

Fees `rpc('record_fee_payment')` DB FN then `POST /api/receipts/generate` API for the PDF.

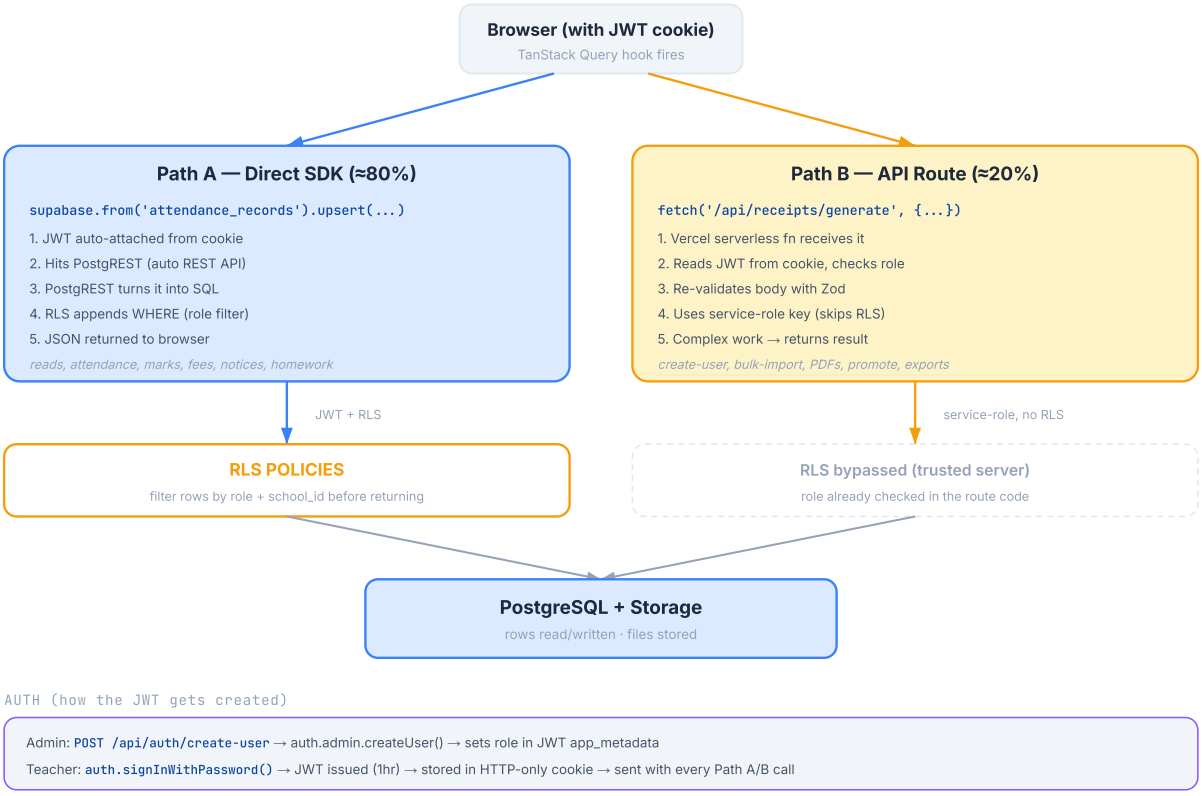
Exams `from('student_marks').upsert()` SDK, `rpc('get_exam_results')` DB FN, `POST /api/reports/bulk-report-cards` API.

Settings / Users `POST /api/auth/create-user` and `PATCH /api/auth/reset-password` API — principal only.

DIAGRAM 03

Backend Layer — the two API paths in detail

How a request is handled. Path A goes straight to the database; Path B goes through a Vercel function first. Both end at RLS.



Backend endpoints — full list with method + role

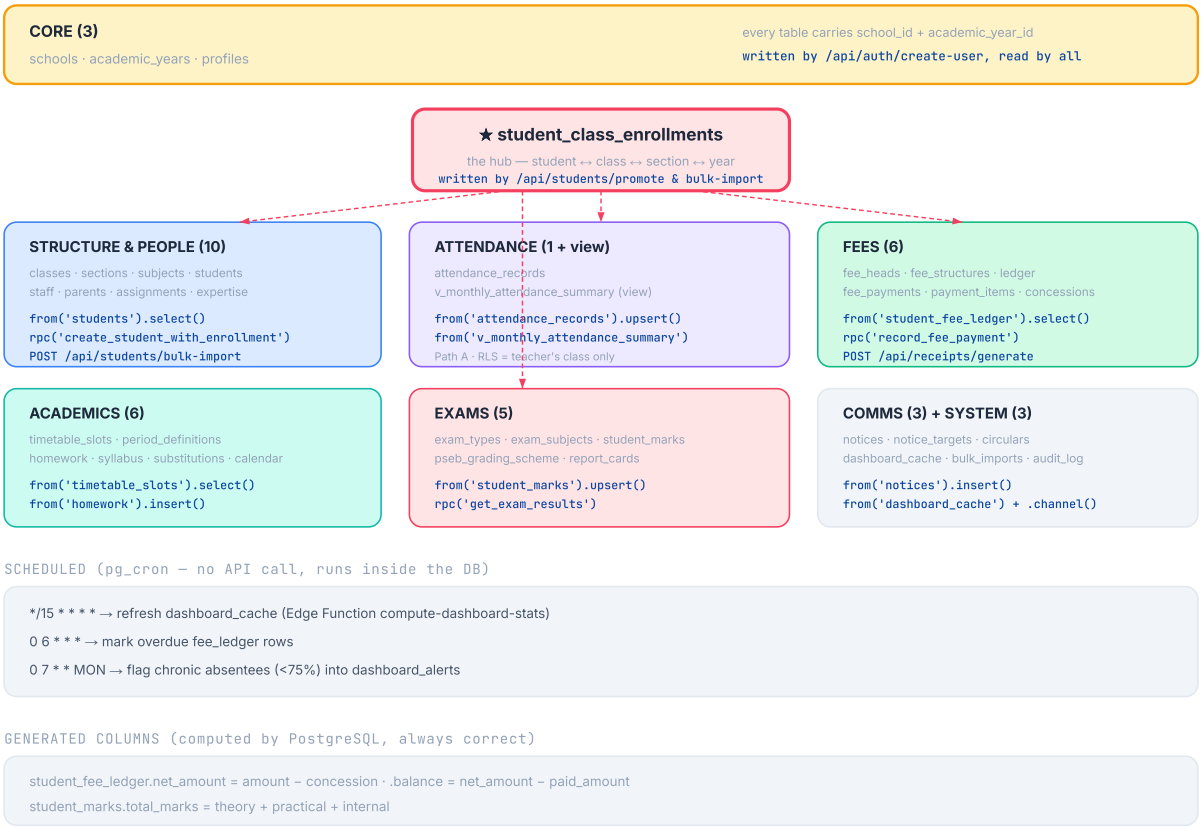
`POST /api/auth/create-user` API	Create account. `principal, clerk`. Uses `auth.admin.createUser()` + inserts profiles/staff.
`PATCH /api/auth/reset-password` API	Reset a user's password. `principal` only.
`POST /api/students/bulk-import` API	Batch insert students from validated JSON. `principal, clerk`.

`POST /api/students/promote`	API	Create next-year enrollments. `principal`.
`POST /api/receipts/generate`	API	Fee receipt PDF → Storage → signed URL. `accountant, principal`.
`POST /api/reports/bulk-report-cards`	API	Queue job → Edge Function generates PDFs → ZIP. `principal`.
`rpc('record_fee_payment')`	DB FN	Payment + payment_items + ledger update in one transaction. `accountant`.
`rpc('get_exam_results')`	DB FN	Totals, grades, ranks computed in SQL. `class_teacher, principal`.
`rpc('create_student_with_enrollment')`	DB FN	Student + parent + enrollment atomically. `principal, clerk`.

DIAGRAM 04

Database Layer — tables and the calls that hit them

30 tables in 9 modules. Each module box lists the API calls that read or write it.



Which API touches which module

Structure & People — `from('students')`, `rpc('create_student_with_enrollment')`, `POST /api/students/bulk-import` & `/promote`.

Attendance — `from('attendance_records').upsert()`, read `from('v_monthly_attendance_summary')`.

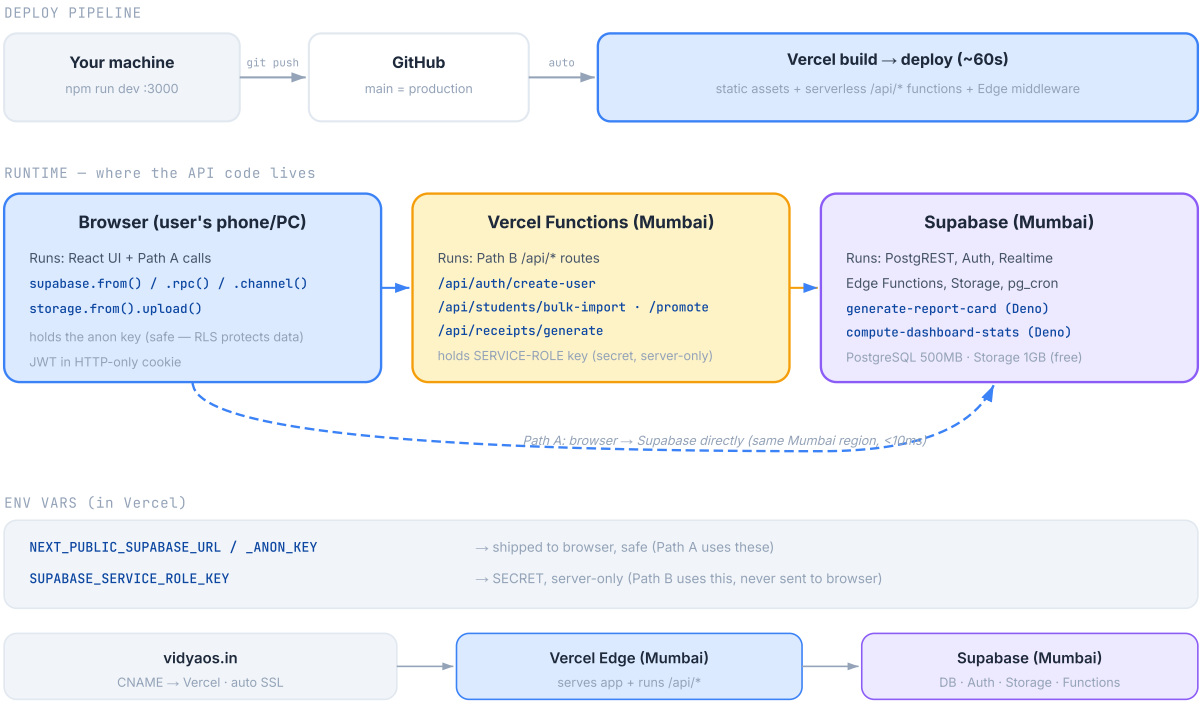
Fees — `from('student_fee_ledger')`, `rpc('record_fee_payment')`, `POST /api/receipts/generate`.

Exams	`from('student_marks').upsert()`, `rpc('get_exam_results')`, `POST /api/reports/bulk-report-cards`.
Academics	`from('timetable_slots')`, `from('homework').insert()`.
Comms & System	`from('notices')` + `notice_targets`; dashboard reads `dashboard_cache` + `.channel()`.

DIAGRAM 05

Deployment Layer — where each API physically runs

Path A code runs in the browser. Path B code runs as Vercel functions in Mumbai. Both reach Supabase in Mumbai.



Where each API runs (and which key it uses)

`Path A calls` SDK	Run in the **browser** with the public anon key. Safe because RLS filters every row by the user's JWT.
`Path B routes` API	Run as **Vercel serverless functions** in Mumbai, holding the secret service-role key that never reaches the browser.
`Edge Functions` DENO	Run **inside Supabase** for heavy async jobs (report-card PDFs, dashboard stats).

`pg_cron` Runs **inside PostgreSQL** — no API call, no server. Refreshes cache, flags defaulters/absentees.